A PROJECT REPORT

FOR

Foundations of Artificial Intelligence

AN AUTONOMOUS AI AGENT FOR GENERATING AND SELF-REPAIRING ROLE-BASED WEB APPLICATIONS FROM NATURAL LANGUAGE

A PROJECT REPORT

Submitted by

2116240701526 – SRI AYYAPPA S

RAJALAKSHMI ENGINEERING COLLEGE

October, 2026

BONAFIDE CERTIFICATE

Certified that this project report "AN AUTONOMOUS AI AGENT FOR GENERATING AND SELF-REPAIRING ROLE-BASED WEB APPLICATIONS FROM NATURAL LANGUAGE" is the bonafide work of "SRI AYYAPPA S" who carried out the project work under my supervision.

SIGNATURE OF THE FACULTY INCHARGE

Submitted for the Practical Examination held on _____

SIGNATURE OF THE INTERNAL EXAMINER

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	3
1	INTRODUCTION	4
2	LITERATURE REVIEW	6
3	PROPOSED SYSTEM	9
	3.1 Architecture Diagram	10
4	MODULES DESCRIPTION	11
	4.1 Natural Language Spec Extractor	11
	4.2 CSP-Based Schema Validator	12
	4.3 Identifier Safety Validator	12
	4.4 Database Schema Generator	13
	4.5 Backend (API + Authorization) Generator	13
	4.6 Frontend Generator	14
	4.7 Smoke Test Generator and Sandbox Runner	14
5	IMPLEMENTATION AND RESULTS	16
	5.1 Experimental Setup	16
	5.2 Results	17
6	CONCLUSION AND FUTURE WORK	20
7	REFERENCES	22

(Page numbers shown are approximate based on this document's current layout. In Word, right-click the table above and select "Update Field" after any edits, or manually adjust numbers to match final pagination before printing.)

ABSTRACT

Building a web application conventionally requires a developer to translate requirements into a database schema, backend logic and access-control rules, and a user interface, and to verify by hand that these three layers agree with one another. This project investigates whether a system can perform this translation automatically for a bounded but practically useful class of software: role-based CRUD (Create-Read-Update-Delete) business applications such as attendance-management, library-management and inventory-management systems, which share a common underlying structure of user roles, data entities and permissions.

The system accepts a natural-language description of the desired application and passes it through a pipeline of five stages. First, a large language model extracts a structured JSON specification of the application's roles, entities, fields and pages; this output is validated against a formal JSON Schema and, on failure, is corrected through a bounded feedback-retry loop. Second, the specification is checked for logical consistency using a Constraint Satisfaction Problem (CSP) formulation, in which entity names, role names and every cross-reference between them are treated as variables constrained to exist within the specification's own domain; this catches errors — such as a field referencing an entity that was never defined — that shape-only validation cannot detect. Third, an identifier-safety check defends against unsafe names

reaching generated code. Fourth, three deterministic, template-based generators produce an SQL database schema, a FastAPI backend whose authorization rules are compiled mechanically from the specification's role-permission data, and a set of static HTML/JavaScript frontend pages. Fifth, a smoke-test generator automatically derives a test case for every (role, action, entity) combination present in the specification, and a sandbox runner executes the generated application against these tests using live HTTP-style requests, exercising the real, running code rather than merely inspecting it.

The system was evaluated by generating two structurally distinct applications — an attendance-management system and a library-management system — from independently written specifications, using the same, unmodified generator code for both. The automatically derived test suites (28 tests for the attendance system and 19 for the library system) confirmed that role-based access control, including scope-restricted "own-record-only" permissions, was genuinely enforced by the generated backends. During development, the exhaustive test suite surfaced two real defects — a database connection-leak causing lock contention under rapid requests, and a test-data collision — both of which were diagnosed and corrected at the generator (template) level, so that every subsequently generated application benefits from the fix. This demonstrates a working, verified instance of the detect-diagnose-repair-reverify loop that characterises an intelligent agent, implemented here using

classical AI techniques — constraint satisfaction and a PEAS-structured agent formulation — alongside large-language-model-based natural language understanding.

CHAPTER 1

INTRODUCTION

Software development for small, internally used business applications — attendance registers, library catalogues, inventory trackers, hostel or canteen management systems — follows a remarkably repetitive pattern. A small number of user roles need to create, read, update and delete records belonging to a small number of data entities, subject to permission rules that determine which role may act on which entity and whether that access is restricted to records the user themselves owns. Despite this repetition, each such application is traditionally designed, coded, secured and tested from scratch, consuming disproportionate developer effort relative to the conceptual novelty involved.

Recent advances in large language models (LLMs) have made it possible to generate source code directly from natural-language descriptions, and commercial tools such as Bolt.new, Replit Agent and Lovable now offer "prompt-to-application" generation for general web and mobile software (see Chapter 2). However, generating code directly from a free-text prompt, with no intermediate structure and no mechanism to verify logical or security properties of the result, is difficult to validate, difficult to modify predictably, and offers no principled way to guarantee that access-control rules implied by the prompt are actually enforced in the generated code.

This project narrows the problem to a well-defined, practically significant subclass of software — role-based CRUD business applications — and builds a pipeline that: (i) converts a natural-language description into a structured, machine-checkable specification; (ii) validates that specification for both shape and logical consistency before any code is written; (iii) deterministically generates a database schema, a backend with mechanically derived

authorization logic, and a frontend from that specification; and (iv) automatically derives and executes tests that verify the generated application's behaviour, closing a detect-diagnose-repair loop characteristic of an intelligent agent rather than a one-shot code generator.

The project deliberately restricts its scope. It does not attempt to generate arbitrary application types, mobile applications, or applications requiring real-time interaction; it targets role-based CRUD applications specifically, because this class of software is both common in practice and tractable enough to generate, validate and test with formal guarantees within the timeframe of an academic project. The remainder of this report surveys related systems and research (Chapter 2), describes the proposed system and its architecture (Chapter 3), details each module (Chapter 4), presents the implementation, experimental setup and results (Chapter 5), and concludes with a discussion of limitations and future work (Chapter 6).

CHAPTER 2

LITERATURE REVIEW

2.1 Commercial Prompt-to-Application Platforms

Bolt.new, Replit Agent and Lovable are the most widely used commercial systems that convert natural-language prompts into working full-stack applications [1], [2]. Bolt.new is a browser-based agent that generates full-stack JavaScript applications and handles code generation, automated testing, and hosting directly in the browser. Replit Agent pairs an autonomous coding agent with a complete IDE and terminal access, targeting both technical and non-technical users with real backend runtime flexibility. Lovable generates a complete full-stack application — a React frontend together with a provisioned PostgreSQL backend, authentication, file storage and payment integration — aimed specifically at taking a prototype through to a shippable product [1].

These platforms share three characteristics relevant to this project. First, they generate code directly from a conversational prompt using a general-purpose LLM, without a mandatory, independently validated intermediate specification of the application's data model and permissions. Second, their internal mechanisms for guaranteeing that generated access-control logic matches the user's stated intent are not published or independently verifiable. Third, they target arbitrary, open-ended application domains, which necessarily limits how deeply any single class of application's correctness can be checked. This project differs by deliberately narrowing scope to one well-understood application family in exchange for a specification layer that is formally validated — via JSON Schema and a Constraint Satisfaction Problem formulation — before a single line of code is generated, and whose generated authorization logic is independently, automatically tested against every role-action-entity combination implied by that specification.

2.2 LLM-Based Automated Program Repair

A substantial and rapidly growing body of academic research addresses Automated Program Repair (APR) using large language models. A systematic literature review of 189 papers on LLM-based APR [3] distinguishes conversational repair, in which a human or external tool supplies feedback that the LLM uses to revise code, from self-repair, in which the LLM investigates its own generated code's execution results and corrects it without external guidance. The same review reports an empirical finding of particular relevance to this project: existing LLMs "often fail to provide reliable, accurate, and valuable feedback on why the code is incorrect" when attempting self-repair, and that self-repair is "not a panacea" for code-generation errors [3]. A complementary taxonomy of 62 LLM-based repair systems [4] classifies repair approaches by the degree of control authority the LLM is given over the repair loop versus externally supplied analysis.

This finding directly informed a key design decision in the present project. Rather than relying on an LLM to freely diagnose and patch its own generated code (the self-repair pattern shown by [3] to be unreliable), this project generates code deterministically from templates, so that the correctness of generated authorization logic depends on the correctness of a small, human-reviewed set of templates rather than on an LLM's variable code-generation quality for each individual application. The "repair loop" demonstrated in this project (Chapter 5) accordingly operates at the template level — a failure detected by automatically generated tests is diagnosed and the underlying template is corrected, after which every subsequently generated application benefits — rather than attempting unreliable, per-application LLM self-repair of already-generated source code.

2.3 Classical AI Foundations Applied

The project draws on two classical AI topics beyond LLM usage. The agent, environment, PEAS (Performance measure, Environment, Actuators, Sensors) and rationality concepts used to frame the overall system (Chapter 3) follow the standard treatment of intelligent agents in AI foundations texts such as Russell and Norvig [5]. The Constraint Satisfaction Problem formulation used for specification validation (Chapter 4.2) — variables, domains and constraints, solved here by constraint checking over an already-formed assignment rather than by backtracking search — likewise follows the standard CSP treatment in the same text [5].

2.4 Research Gap Addressed by This Project

Commercial platforms [1], [2] demonstrate that natural-language-to-application generation is practically achievable at scale but do not publish verifiable mechanisms for guaranteeing that generated access-control logic is correct, and target arbitrary domains rather than a formally specified subclass. Academic APR research [3], [4] demonstrates that unconstrained LLM self-repair of generated code is not yet reliable. The gap this project addresses sits between these two bodies of work: a narrowly scoped, template-based generator whose specification layer is formally validated using classical AI techniques (schema validation and CSP checking) before generation, and whose generated output is verified not by LLM self-assessment but by exhaustively and automatically derived tests executed against the real, running application.

CHAPTER 3

PROPOSED SYSTEM

The proposed system is framed as an intelligent agent in the PEAS sense: its environment is the specification and the generated codebase; its sensors are the outputs of validation checks and automatically executed tests; its actuators are the code-generation and template-correction operations it performs; and its performance measure is the proportion of automatically derived tests that pass against the generated application. The system operates in five stages, described briefly here and in full detail in Chapter 4.

Stage one accepts a natural-language application description and uses an LLM to produce a structured JSON specification comprising the application's roles, their permissions, its data entities and fields, and its pages. Stage two validates this specification in two steps: JSON Schema validation confirms the specification is well-formed, and a Constraint Satisfaction Problem solver confirms it is logically self-consistent — every cross-reference between roles, entities and pages resolves to something that actually exists in the specification. Stage three deterministically generates an SQL database schema, a FastAPI backend whose authorization checks are compiled directly from the specification's permission data, and a set of static frontend pages. Stage four automatically derives a smoke-test suite from the same specification and executes it against the generated, running application inside an isolated process sandbox. Stage five, triggered when a test fails, diagnoses the failure and corrects the responsible generator template, after which the pipeline regenerates and re-tests, bounded to a small number of attempts.

A deliberate design decision underlies the whole pipeline: code generation is template-based rather than free-form LLM generation. This choice trades some generative flexibility for a guarantee that is difficult to obtain otherwise

— that every generated application's access-control logic is produced by the same small, independently reviewed and tested set of generation rules, rather than being re-synthesised, with variable reliability, by an LLM for every individual application.

3.1 Architecture Diagram

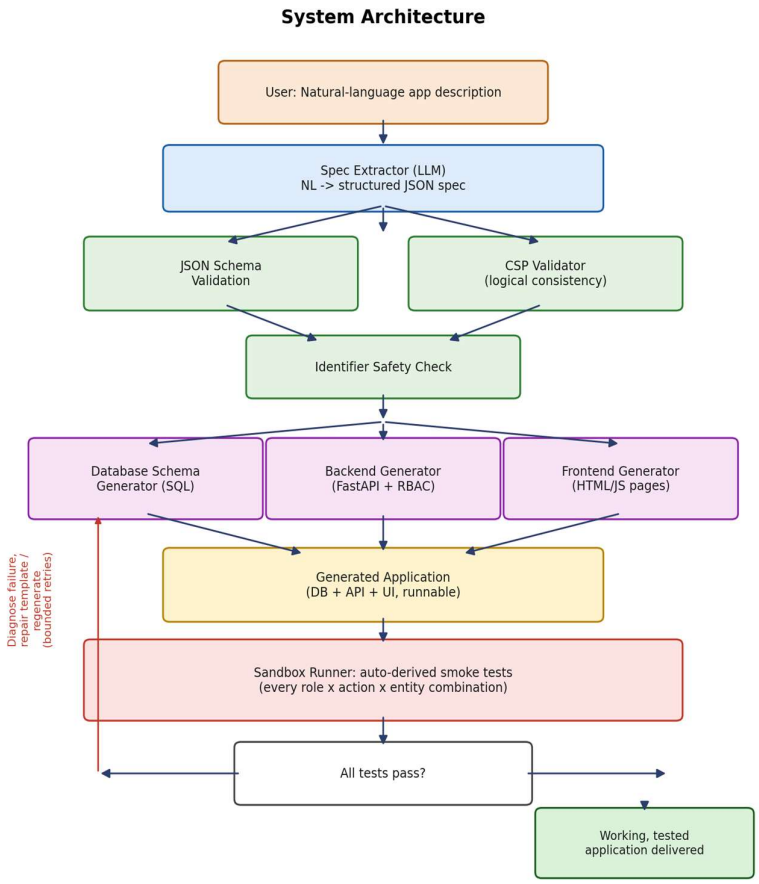


Figure 3.1: End-to-end system architecture, from natural-language input to a tested, running application.

CHAPTER 4

MODULES DESCRIPTION

4.1 Natural Language Spec Extractor

This module converts the user's plain-English description into a structured JSON specification using an LLM constrained, via a strict system prompt, to output only JSON matching a predefined schema. The output is parsed and validated against a formal JSON Schema (required fields, correct types, non-empty arrays for roles, entities and permissions). If validation fails, the specific validation error is returned to the LLM together with an instruction to correct its own output, and the call is retried up to three times before the extraction is reported as failed rather than allowing malformed data to proceed. Every extraction attempt is logged for later evaluation. For development and testing, in the absence of live API access (Section 5.1), this module also supports a mock mode that returns a deterministic, realistic specification without an external API call, allowing every downstream module to be built and verified independently of external service availability.

4.2 CSP-Based Schema Validator

A specification that is shape-valid under JSON Schema may still be logically inconsistent — for example, a field may declare a reference to an entity named "Studnet" that does not exist anywhere else in the specification. This module models the problem as a Constraint Satisfaction Problem: the variables are every cross-reference in the specification (a field's referenced entity, a permission's target entity, a page's accessible-by role list, a page's related entity); the domain of each variable is the set of entity or role names actually declared in the specification; and the constraints require every variable's value to belong to its corresponding domain, together with uniqueness constraints on entity and role names. The validator performs constraint checking over the

fully formed specification (it verifies a candidate assignment rather than searching for one via backtracking) and reports every violation found, rather than stopping at the first, so that a developer or the retry loop receives a complete diagnostic in one pass.

4.3 Identifier Safety Validator

As defence in depth beyond CSP validation, this module checks that every name that will be inserted into generated SQL or Python source (entity names, field names, role names) matches a safe identifier pattern and is not a Python or SQL reserved word, before any code generation is attempted. This treats every specification value as untrusted input regardless of its origin, consistent with standard secure-coding practice for any system that inserts externally supplied names into generated code or queries.

4.4 Database Schema Generator

This module deterministically translates each validated entity into a SQL CREATE TABLE statement for SQLite. Every table receives an auto-incrementing integer primary key; reference-type fields become INTEGER foreign keys; required and unique constraints declared in the specification are carried over directly into NOT NULL and UNIQUE column constraints. The generated SQL was verified by executing it against a live SQLite database and performing real inserts and reads, rather than only inspecting the generated text.

4.5 Backend (API + Authorization) Generator

This is the highest-stakes module in the system, since an error here does not merely break a build but can silently produce an insecure application. For every entity, the generator scans every role's declared permissions and generates a FastAPI endpoint only for an (action, entity) combination that at

least one role is actually granted; no endpoint is generated, and hence none can be called, for an action no role is permitted to perform. Every generated endpoint checks the requesting user's role against the exact set of roles the specification grants for that action before executing any database operation. Where a permission's scope is "own" rather than "all", the generator locates the field on the entity that references the requesting role's underlying entity (for example, an Attendance record's reference to Student, for the Student role) and uses it to restrict both which rows are returned and which rows may be modified or deleted to the requesting user's own records; if no such unambiguous ownership field exists, the generator refuses to produce that endpoint rather than silently granting broader access than intended. Authorization enforcement was verified not by inspection but by actually running the generated application and issuing live requests (Section 5.2).

4.6 Frontend Generator

This module produces one static HTML page per page declared in the specification, plus a login page, using vanilla JavaScript with no build step. Each page's JavaScript embeds the list of roles permitted to view it and redirects to the login page if the role held in the browser's local storage is not in that list; this client-side check exists purely for user experience and is explicitly not the system's security boundary, since the backend's own authorization checks (Module 4.5) cannot be bypassed by editing client-side code. Pages that correspond to an entity on which the viewing role holds create permission additionally render a data-entry form whose fields are derived directly from that entity's specification.

4.7 Smoke Test Generator and Sandbox Runner

This pair of modules implements the "observe results" component of the agent loop. The smoke-test generator automatically derives one test case for every

(role, action, entity) combination implied by the specification: a positive case asserting that a role which holds the corresponding permission receives HTTP 200, a negative case asserting that every role which does not hold that permission receives HTTP 403, and an anonymous case asserting that a request with no authentication token receives HTTP 401. The sandbox runner seeds minimal data for each entity, then imports and actually executes the generated FastAPI application in an isolated working directory with a fresh database file, issuing real HTTP-style requests via FastAPI's TestClient and recording the pass/fail outcome and diagnostic detail for every derived test. This sandboxing is lightweight — isolated process and working directory rather than full OS-level container isolation — a deliberate choice justified by the fact that the code under test is deterministically template-generated rather than freely LLM-authored (Section 6.2 discusses this limitation and its implication for future work).

CHAPTER 5

IMPLEMENTATION AND RESULTS

5.1 Experimental Setup

The system was implemented in Python. The technology choices and their justification are summarised below.

- Backend generation target: Python with FastAPI, chosen for low boilerplate and built-in request validation.
- Database: SQLite, a file-based database requiring no separate server process, appropriate for a generated application of this scale.
- Frontend generation target: static HTML with vanilla JavaScript, avoiding a Node.js/build-step dependency for the generated output.
- Specification validation: the jsonschema Python library for shape validation, and a custom-built CSP checker (Section 4.2) for logical consistency.
- Natural language extraction: the Gemini API (google-genai SDK) was the intended LLM provider. At the time of writing, the Gemini free tier returned a persistent 429 RESOURCE_EXHAUSTED / limit:0 error under two independently created accounts, most consistent with a regional access restriction rather than a code defect. Consequently, Module 4.1 was developed and verified using a deterministic mock-extraction mode (Section 4.1); the live-call code path is complete, unit-testable, and requires no further code changes once API access is restored.
- Testing: FastAPI's TestClient was used throughout to issue genuine HTTP-style requests against running instances of generated applications, rather than inspecting generated source code for plausibility.

5.2 Results

Two structurally different applications were generated from independently authored specifications using the identical, unmodified generator code: an attendance-management system (roles: Student, Faculty, Admin; entities: Student, Faculty, Subject, Attendance) and a library-management system (roles: Member, Librarian; entities: Member, Book, Loan). Table 5.1 summarises the automatically derived smoke-test results for both.

Generated Application	Auto-Derived Smoke Tests	Result
Attendance Management System	28	28 / 28 passed
Library Management System	19	19 / 19 passed

Table 5.1: Auto-derived smoke test results across two independently specified applications.

These results confirm, by executing real requests against the running generated applications rather than by inspection, that: a role granted a permission receives the expected success response; a role not granted that permission is rejected with HTTP 403; an unauthenticated request is rejected with HTTP 401; and, for permissions scoped to "own" records, a user retrieving a list of records receives only the records that belong to them — verified concretely in the attendance system, where a Student user issuing a read request received exactly one attendance record, corresponding to their own, rather than records belonging to another student.

During development of the automatically derived, exhaustive test suite (every role-action-entity combination, as opposed to the smaller, hand-selected set of cases checked in earlier manual testing), two genuine defects were surfaced on the first exhaustive run against the attendance system. Both were diagnosed to a root cause and corrected at the generator-template level rather than patched per application, so that every application generated afterward, including the independently specified library system, is unaffected by either defect. Table 5.2 summarises these.

Defect	Root Cause	Resolution
Database lock contention under rapid requests	Generated database.py opened a new SQLite connection per request without closing it	Added try/finally connection closure to every generated endpoint (template-level fix)
False UNIQUE constraint failure on update test	Auto-derived create and update smoke tests used identical sample values for a unique field	Smoke-test generator modified to produce distinct sample values per test case

Table 5.2: Defects surfaced by exhaustive automated testing, with root cause and template-level resolution.

After both corrections, the full 28-test suite for the attendance system and the full 19-test suite for the library system passed without modification to any application-specific code, confirming the fixes were genuinely general rather than specific to one generated application. This detect-diagnose-correct-reverify sequence, executed against real running code rather than simulated or assumed, is the concrete evidence offered by this project for the "self-repairing" component of its stated objective, implemented at the generator level in line with the template-based design justified in Section 2.2.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 Conclusion

This project demonstrates a working pipeline that converts a natural-language description of a role-based CRUD business application into a generated, running application whose database schema, backend authorization logic and frontend are mechanically derived from a specification that is itself validated for both structural correctness (JSON Schema) and logical consistency (a Constraint Satisfaction Problem formulation) before any code is produced. The generated application's correctness — specifically, that its access-control logic genuinely matches the permissions declared in the specification — was verified not by inspection but by automatically deriving an exhaustive test suite from the same specification and executing it as live requests against the running generated application, for two independently specified, structurally distinct applications, using identical generator code. The exhaustive test suite surfaced two genuine defects during development; both were diagnosed and corrected at the generator-template level, demonstrating a real, verified instance of the detect-diagnose-repair-reverify cycle that distinguishes an intelligent agent from a one-shot generation pipeline.

The project also makes deliberate, documented scope decisions rather than claiming unqualified generality: it targets role-based CRUD applications specifically rather than arbitrary software; its authentication mechanism is explicitly demo-grade rather than production-grade; its sandboxing is a lightweight, isolated-process design rather than full OS-level container isolation, justified by its template-based (rather than free-form LLM-generated) code; and its natural-language extraction module, while fully implemented and unit-tested, operated in a deterministic mock mode during

evaluation due to an external API access restriction outside the project's control.

6.2 Limitations

• Live large-language-model extraction (Module 4.1) could not be evaluated end-to-end due to Gemini API free-tier access restrictions encountered during development; the code path is complete but unverified against a live model at the time of writing.

• The authentication scheme used by generated applications is intentionally minimal, for demonstrating and testing role-based access control, and is explicitly not suitable for production deployment without replacement by a real authentication system.

• Sandboxing is process-level rather than container-level, which is an appropriate but not unconditional trade-off tied to the generator remaining template-based.

• The scope is deliberately restricted to role-based CRUD applications; the system does not generate arbitrary application types, real-time applications, or mobile applications.

6.3 Future Work

• Restore and evaluate live Gemini API extraction once access is available, and benchmark extraction accuracy across a larger and more varied set of natural-language prompts.

• Implement the modification pathway (editing an existing generated application's specification in response to a follow-up natural-language instruction, re-validating with the same CSP and identifier-safety checks, and regenerating only the affected code), which is designed but not yet implemented.

- Replace the demo-grade authentication module with a production-appropriate implementation (hashed credentials, signed tokens).
- Migrate sandboxing to container-based isolation (for example, Docker) if the code-generation approach is extended beyond pure templating toward any LLM-authored code.
- Extend the heuristic/local-search component of the repair loop to rank multiple candidate template fixes when more than one plausible correction exists, drawing more fully on the heuristic search material introduced in Unit II of the course syllabus.

CHAPTER 7

REFERENCES

[1] Lovable Team (2026) 'Bolt vs Replit vs Lovable: Which AI App Platform Wins?', Lovable Guides. Available at: https://lovable.dev/guides/bolt-vs-replit-vs-lovable

[2] Automation Atlas (2026) 'Replit Agent vs Lovable vs Bolt.new: which AI app builder should you use in 2026?'. Available at: https://automationatlas.io/answers/replit-agent-vs-lovable-vs-bolt-new-2026/

[3] Zhang, Q. et al. (2024) 'A Systematic Literature Review on Large Language Models for Automated Program Repair', arXiv:2405.01466.

[4] Yang, B. et al. (2025) 'A Survey of LLM-based Automated Program Repair: Taxonomies, Design Paradigms, and Applications', arXiv:2506.23749.

[5] Russell, S. and Norvig, P. (2021) Artificial Intelligence: A Modern Approach, 4th edn. Pearson.

[6] FastAPI Documentation. Available at: https://fastapi.tiangolo.com

[7] SQLite Documentation. Available at: https://www.sqlite.org/docs.html

[8] Google AI for Developers, Gemini API Documentation. Available at: https://ai.google.dev/gemini-api/docs

[9] JSON Schema Specification. Available at: https://json-schema.org